BE[CSE] – 7nd Sem Minor Project (Dec 2025)

ShadowTrace

---

1. Title

ShadowTrace – Real-Time, Extensible macOS Observability & Control (Rust & C Backend)

ShadowTrace is a macOS application that gives you real-time visibility and control over every corner of your system—CPU, memory, disk, network, and file activity—powered by lean Rust and C code for minimal overhead. Through five progressive phases—live monitoring, automated plugin reactions, detailed system-call tracing, instant configuration changes, and a full developer toolkit with log exports and an easy-install Mac package—ShadowTrace makes it effortless to watch, respond to, and extend your Mac's behavior.

---

2. Brief Introduction

ShadowTrace is a macOS app that watches everything your computer does, from which programs are running to which files are changing, all with extremely fast, low-overhead code in Rust and C—no Python involved. It rolls out in five steps:

1. **Basic Monitoring:** Continuously track CPU, memory, disk, network, and file activity in real time.
2. **Automated Responses:** Let small "plugins" react to important events—like sending you an alert when a program uses too much memory.
3. **Deep Tracing:** Record detailed system calls and file changes to help you diagnose exactly what happened and when.
4. **Custom Configuration:** Use a simple settings file to turn any part of the monitor or plugin system on or off without rebuilding the app.
5. **Developer Toolkit & Exports:** Provide a ready-made kit for other programmers to write their own plugins, plus easy export of logs and dashboards, and a fully packaged, signed Mac installer.

---

3. Aim

1. **Phase I (“The Eye”)**
    - Write **Rust** daemons for CPU, memory, disk, and network metrics using sysinfo and IOKit bindings in C.
    - Implement **C**–based FSEvents watcher for file-I/O, exposing a zero-copy ring buffer.
    - Serve JSON-RPC over Unix sockets with a Rust + tokio server.
2. **Phase II (“The Hand”)**
    - Embed a **Rust** plugin engine loading .dylib modules, exposing a C ABI for plugins in Swift or C.
    - UI to register triggers; plugins can be written in Rust or plain C.
3. **Phase III (“The Engine”)**
    - Enhance file tracing throughput with a C ring-buffer queue integrated via FFI into Rust collector.
    - Implement remediation hooks entirely in Rust (e.g., std::process::Command to run shell/AppleScript).
4. **Phase IV (“The Mesh”)**
    - Package all Rust and C modules into a single Xcode project.
    - Parse shadowtrace.yaml config in Rust using serde_yaml; no Python.
    - Dynamically enable/disable collectors and plugins at runtime.
5. **Phase V (“The Mind”)**
    - Provide a **Rust**-based plugin SDK (Cargo templates) and **C** header files.
    - Generate local API docs with rustdoc + Doxygen for C.
    - Add CSV/PNG export implemented in Rust (csv crate) and C (libpng).
    - Produce a signed, notarized .dmg bundling SwiftUI front-end and Rust/C backend.

---

4. Tools & Technologies

- **Rust Crates:** sysinfo, tokio, serde/serde_yaml, jsonrpc-core, csv
- **C Libraries & APIs:** IOKit for CPU/memory, DiskArbitration, FSEvents API, libpng, POSIX APIs
- **FFI & Interop:** Rust’s cc crate for building C modules; #[repr(C)] for structs; dlopen/dlsym for plugin loading
- **Frontend:** Swift 5 + SwiftUI; JSON-RPC client over Unix sockets

- **Configuration:** shadowtrace.yaml parsed in Rust (serde_yaml); JSON Schema validation via Rust types
- **Packaging & Signing:** Xcode project integrating Rust/C builds via cargo build --release + custom Run Script; codesign, notarytool, create-dmg

---

5. Expected Outcome

- **Phase I:** Sub-10 ms sampling daemons for CPU, RAM, disk, network; FSEvents watcher pushing events into Rust ring buffer; overall backend < 2 % CPU.
- **Phase II:** Plugin panel in UI driving Rust .dylib or C plugins; trigger actions and log to ~/Library/Logs/ShadowTrace.
- **Phase III:** High-throughput file event queue (> 10 k events/s) without drop; remediation commands executed from Rust handlers.
- **Phase IV:** Single macOS .app loading config at startup; hot-toggle collectors/plugins per YAML without restart.
- **Phase V:**
  - **SDK:** Cargo template project and C header + sample.
  - **Docs:** rustdoc HTML + Doxygen output in Documentation/.
  - **Exports:** CSV via Rust csv crate; PNG snapshots via libpng calls in C.
  - **Installer:** Signed, notarized .dmg with silent-install support.

---

6. Proposed Logo (Initially)

7. Gantt Chart

Weeks	Dates	Activity (Rust & C Backend)
1–2	04 Aug–17 Aug	**Phase I**: Build Rust daemons (sysinfo, IOKit bindings), C FSEvents watcher, RustNIO JSON-RPC server
3–4	18 Aug–31 Aug	**Phase II**: Rust plugin engine; UI for loading .dylib/C modules; logging framework
5–6	01 Sep–14 Sep	**Phase III**: Integrate C ring buffer queue; implement remediation handlers in Rust
7–9	15 Sep–05 Oct	**Phase IV**: Merge Rust/C builds into Xcode; serde_yaml config loader; dynamic module toggling
10–12	06 Oct–26 Oct	**Phase V**: Create Rust & C SDK templates; generate docs (rustdoc + Doxygen); implement CSV/PNG export; package
13–14	27 Oct–09 Nov	**Testing & Optimization**: 72 h stability; backend CPU < 2 %; plugin stress tests
15	After Minor 2	**Mid-Term Evaluation** (25 %)
16–18	10 Nov–30 Nov	**Final Bug-Fix & Demo Prep**: polish UI, finalize slides, rehearse
19	After Minor 2	**Final External Evaluation** (25 %)

---

8. Team Members & Roles

1. Tikshananshu

Role – System Architect and Designer

Responsibilities:

- Design the observability core (syscall tracing, PID tracking, resource telemetry)
- Interface with fs_usage, proc_pidinfo, sysctl, auditd, and vm_stat
- Handle core logging, error recovery, and performance tuning
- Design the internal data schema (syscall logs, file events, plugin triggers)

Skillset Required:

- C / C++ / Rust
- Experience with macOS tracing tools (DTrace, fs_usage, libproc)
- Familiarity with Mach APIs, task_info, sysctl trees
- Strong focus on performance, resilience, and debugging

---

2. Nihal Pandey

Role – Backend Engineer and Designer

Responsibilities:

- Build the plugin engine (trigger–response architecture)
- Design plugin API schema and dynamic loader
- Implement sandboxing, plugin crash isolation, resource-limited execution
- Connect plugins to core triggers (syscall, memory, file, network)

Skillset Required:

- Rust or C++
- Python (for rapid prototyping of the plugin API)
- Multiprocessing, IPC, subprocess control
- Familiarity with sandbox-exec, UNIX pipes, plugin packaging

---

3. Simarpreet Singh

Role – UI/UX Designer

Responsibilities:

- Build the native SwiftUI dashboard interface
- Integrate Combine and observable data flows from the backend
- Implement charts (using Swift Charts/CoreAnimation), memory & network overlays, Secure Mode display
- Handle export controls, search, and PID-tree rendering

Skillset Required:

- Swift, SwiftUI, Combine
- Data-driven animation with CoreAnimation or Swift Charts
- UI optimization for 60 FPS under load
- File I/O and log-viewing integration for plugin/event exports

---

B. Collective Responsibilities

All team members will:

- **Version Control:** Use Git (GitHub/GitLab), commit daily, branch per subsystem
- **Testing:** Join weekly integration tests (trace recovery, plugin injection, export correctness)
- **Documentation:** Maintain inline docs, comment all plugin logic, draft final report sections
- **Internal Reviews:** Rotate Friday code reviews of each other's modules
- **Bug Triage:** Keep a shared bug/issue tracker for crashes or permission failures
- **Stress Testing:** Alternate building ≥ 2 hr monitoring sessions to validate stability

---

C. Collaboration Protocol

- **IDEs:**
  - Xcode (UI, Swift)
  - VSCode or Vim (C/Rust backend, plugin core)
- **Communication:**
  - Shared Notion/GitHub board for issues
  - Weekly report & demo reviews (Saturdays)
  - Plugin registry with signature, trigger, action metadata
- **Development Model:**
  - Subsystem branches (tracer-core, dashboard-ui, plugin-engine)
  - Merge to main only after passing CI/tests
- **Deployment Order:**

  1. CLI-based syscall tracer + resource monitor
  2. Plugin engine with 3 base plugins
  3. SwiftUI dashboard with live feeds
  4. Sensor/network/clipboard layer
  5. Secure Mode, export engine, documentation
